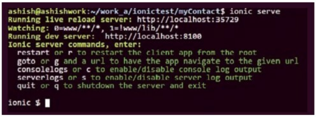

Figure 3: Ionic prompt

it on the emulator. In a terminal, execute the following command:

```
$ cordova platform add android
$ ionic add platform android
$ ionic build android
```

The last command will start the Android emulator and will run the app, as shown in Figure 4.

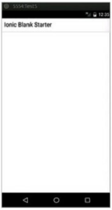

Figure 4: Output of *myContact* on the emulator

Let us explore the *myContact* folder, where we can see many files and folders (Figure 5).

Let's quickly understand what each file and folder is used for.

- *bower.json* stores the bower dependencies
- *config.xml* stores the configuration for Cordova
- *gulpfile.js* stores the gulp task
- *hooks* stores the Cordova hooks for executing a specific command
- *ionic.project* stores the Ionic configuration
- *package.json* stores the dependencies for Node
- *platforms* stores the mobile platform, i.e., Android, iOS, etc
- *plugins* stores the plugins
- *scss* stores and is used in CSS
- *www* is the application folder which stores HTML, CSS, JS, etc

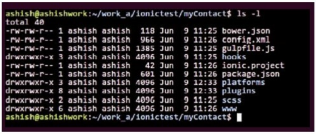

Figure 5: Listing of *myContact* folder

Let us change the application to show some output. We will do it in two steps — first displaying some static data, and then displaying dynamic data. In the *www* folder, we can see *index.html* generating our output, and in the *js* folder, *app.js* is used for interaction. Open the *index.html* and *app.js* files in editing mode [edit, Vim, Notepad, etc]. First, we will change the header text. In *index.html,* locate the body tag and replace the text *Ionic Blank Starter* with *My Contacts*. Locate the *<ion-content>* tag, and add the following code after the tag:

```
<ion-list>
      <ion-item ng-repeat="contact in contacts">
        {{contact.title}}
      </ion-item>
</ion-list>
```

Just to be clear, we are using *ng-repeat* of AngularJS, which is nothing but a kind of loop and will print all the contacts titles on the page.

In the *app.js* file, add the following line after *angular.module('starter', ['ionic']):*

```
.controller('MyContactCtrl', function($scope) {
  $scope.contacts = [
    { title: 'Ashish Bhatia' },
    { title: 'Friend 1' },
    { title: 'Friend 2' },
    { title: 'Friend 3' }
  ];
})
```

We have defined a controller named *MyContactCtrl* and created a contact with some data. We need to add this controller to *index.html*. In the *index.html* file, add the controller in the body tag, which will look like *<body ng-app="starter" ng-controller="MyContactCtrl">*. Save both the files, *index.html* and *app.js.*

In a terminal window, execute the command *ionic serve*, which will open the browser as shown in Figure 6.

When you move back to the terminal, you will see the Ionic command prompt. Press *q* to return to the normal prompt. To test this on an emulator, execute the following commands:

```
$ ionic build android
$ ionic emulate android
```

The last command will launch the emulator, as shown in Figure 7.

The data displayed is static as we have hard coded it in the *js* file. To make it dynamic, we need to add a button for *New*, which will display an input form for the title and will display it in the list.

Again, open the *index.html* and *app.js* files for editing.

In the *index.html* header part where we have added *My Contact* in the *h1* tag, add a button for *New* as shown below:

```
<button class="button button-icon" ng-click="newContact()">
        <i class="icon ion-compose"></i></button>
```

This will create a new button option in the header part. We need to create the input form for accepting user inputs.